SOFTWARE ENGINEERING LAB

Functional Test Case Design

kvstore

Distributed, Transactional and Replicated Key–Value Store



Aim
To identify testable functional requirements from the kvstore SRS, prepare a register containing positive and negative test cases, document detailed test procedures, and verify requirement traceability using observable expected results and the implementation/test files supplied with the project.

Name	Neha Nupur
Roll Number	20251651064
Programme	MCA, 3rd Semester
Subject Code	CS545
Subject	Software Engineering
Academic Session	2026

Prepared with reference to the Lab 9 Functional Test Design slides and the supplied kvstore Software Requirements Specification.

Contents

1 Aim **2**

2 Project Overview and Testing Scope **2**

2.1 Project summary . . . 2

2.2 Scope selected for this lab . . . 2

2.3 Functional testing approach . . . 2

3 Requirements Selected from the SRS **3**

4 Part A — Test Case Register **3**

5 Part B — Detailed Test Case Records **5**

5.1 TC-01: Create a key and read it back (positive test) . . . 5

5.2 TC-14: Commit a cross-shard transaction atomically (positive test) . . . 6

6 Requirement Traceability Matrix **7**

6.1 Traceability checklist . . . 8

7 Test Execution Record and Result **9**

7.1 Execution summary template . . . 9

8 Conclusion **9**

1 Aim

To design a systematic functional test set for the `kvstore` project by selecting requirements from its SRS, defining test inputs and preconditions, writing repeatable execution steps, specifying clear expected results, including both positive and negative scenarios, and mapping every test case to one or more requirement IDs.

2 Project Overview and Testing Scope

2.1 Project summary

`kvstore` is a distributed, transactional and replicated key–value store written in Go. Its core design includes Raft consensus and replication, a storage engine with MVCC, write-ahead logging (WAL), a leader-seeking client, fault testing, metrics, operational intelligence and optional TLS. The SRS also defines target interfaces such as REST/JSON, Server-Sent Events (SSE), a dashboard and a shard gateway.

2.2 Scope selected for this lab

This assignment focuses on functional behavior that can be checked from observable inputs and outputs. The test register includes core key operations, transaction conflict handling, durability/recovery, leader discovery, metrics, safe operational queries, TLS certificate handling, REST behavior, and shard-boundary validation. The supplied source archive contains implementations for REST/JSON, SSE, the dashboard, metrics, TLS, operational AI, sharding, and a dedicated cross-shard transaction coordinator. Tests are therefore framed around the implemented behavior and the distinction between ordinary single-shard transactions and the dedicated cross-shard endpoint.

Status convention. The supplied repository contains implementations for the features covered here and a corresponding Go test suite. Therefore, the register records implementation status and repository test coverage, not a claim that every manual test in this report was freshly executed during preparation. The project README reports that its full test suite, including race testing, passed previously. A fresh rerun in this environment was not possible because Go 1.25.0 was unavailable locally and the toolchain download was blocked by network access.

2.3 Functional testing approach

Functional testing checks what the system does rather than its internal code structure. Each test has a unique ID, requirement ID, preconditions, inputs, repeatable steps, expected observable outcome, test polarity and execution status. Positive tests exercise valid behavior; negative tests exercise invalid requests, conflicts, unavailable services or disallowed actions.

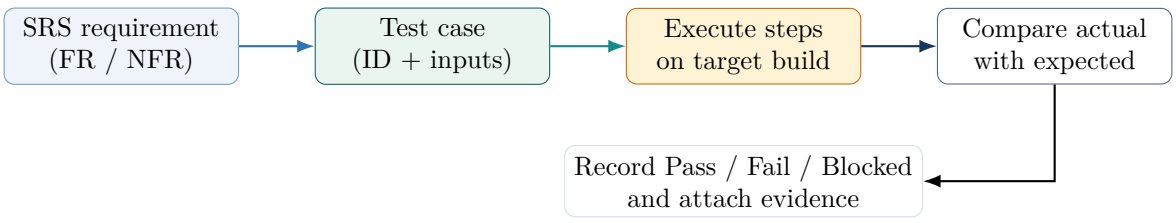


Figure 1: Functional testing workflow used for the kvstore test set.

3 Requirements Selected from the SRS

The following requirements are selected from the supplied SRS. The IDs and meanings are preserved to keep the tests traceable to the original specification.

Req. ID	Requirement under test	Test focus	Observable evidence
FR-02	A healthy cluster elects a valid leader per term.	Leader availability	Node status / election log identifies one leader for the term.
FR-03	The leader replicates committed operations to followers.	Replication	A committed value is visible after follower restart/rejoin.
FR-04	The system returns a key value and found/not-found state.	Read behavior	Existing key returns its value; missing key returns not-found.
FR-05	The system creates or updates a key.	Write behavior	A subsequent read returns the written value.
FR-06	The system deletes a key.	Delete behavior	Subsequent read reports the key as not found.
FR-07	Multiple KV operations execute in one-shard transaction.	Atomic transaction	Commit result and final values match the transaction.
FR-08	MVCC uses snapshot isolation and first-committer-wins conflict detection.	Conflicting writes	One conflicting commit is rejected; no unsafe partial result.
FR-10	Committed state is recoverable from CRC-checked WAL records.	Durability	Committed value survives node restart.
FR-12	Client discovers the leader and retries when the initial node is not leader.	Redirect / retry	Request through follower eventually receives correct result.
FR-16	Metrics are rendered in Prometheus text format.	Metrics export	Response contains expected metric names and valid text format.
FR-21	LLM actions are restricted to rule-engine-whitelisted operations.	Operational query safety	Unsupported command is rejected without executing it.
FR-22	TCP optionally supports TLS/mTLS when configured.	Certificate validation	Valid certificate connects; untrusted certificate is rejected.
FR-23	Target system exposes JSON endpoints for KV, transactions, status, metrics and AI.	REST API	Correct status code, content type and JSON body.
FR-28	Gateway rejects transactions whose operations map to multiple shards.	Cross-shard guard	Request is rejected before forwarding; no partial mutation.

4 Part A — Test Case Register

The register contains more than the minimum six cases and includes both positive and negative tests. The expected results are stated so another tester can determine objectively whether each test passed.

Register A1 — Scenario and classification

Test ID	Req. ID	Scenario / input	Type	Status
TC-01	FR-05, FR-04	PUT a new key `course` with value `MCA`.	Positive	Implemented
TC-02	FR-04	GET a key that does not exist.	Negative	Implemented
TC-03	FR-06, FR-04	DELETE an existing key, then GET it.	Positive	Implemented
TC-04	FR-06	DELETE a key that is already absent.	Negative	Implemented
TC-05	FR-07	Commit a transaction containing two writes to keys in the same shard.	Positive	Implemented
TC-06	FR-08	Two transactions concurrently update the same version.	Negative	Implemented
TC-07	FR-10	Restart a node after a committed write.	Positive	Implemented
TC-08	FR-12, FR-02	Send a client request to a follower.	Positive	Implemented
TC-09	FR-03	Rejoin a follower after a committed write.	Positive	Implemented
TC-10	FR-16	Request Prometheus metrics.	Positive	Implemented
TC-11	FR-21	Submit an unrecognized natural-language query.	Negative	Implemented
TC-12	FR-22	Connect using an untrusted TLS certificate.	Negative	Implemented
TC-13	FR-23	Send a valid REST PUT request.	Positive	Implemented
TC-14	FR-73–FR-78	Submit an atomic cross-shard transaction through the dedicated endpoint.	Positive	Implemented
TC-15	FR-72	Submit a cross-shard request through the ordinary transaction endpoint.	Negative	Implemented

Register A2 — Execution and expected result

Test ID	Execution summary	Expected result
TC-01	Write the key, then read it back.	Write succeeds; GET returns exactly `MCA`.
TC-02	Query a unique absent key.	System returns not-found; it does not fabricate a value.
TC-03	Create key, delete it, then GET.	Delete reports success; subsequent GET is not-found.
TC-04	Delete a unique missing key.	Defined not-found/idempotent behavior is returned; cluster remains healthy.
TC-05	Submit transaction and read both keys.	Commit succeeds; both values are visible together.
TC-06	Start from same snapshot; attempt both conflicting commits.	First valid commit succeeds; conflicting commit is rejected or aborted; no lost update.
TC-07	PUT, confirm commit, stop and restart node, then GET.	Committed value is recovered from durable state.
TC-08	Issue valid operation through a non-leader endpoint.	Client discovers/retries leader and returns the correct operation result.
TC-09	Commit a write, restart follower, inspect replicated state.	Follower catches up; committed value matches the leader's state.
TC-10	Call metrics export and inspect response.	Response includes expected metric names in Prometheus text format.

Test ID	Execution summary	Expected result
TC-11	Submit an unrecognized phrase to the NL query interface.	The rule engine returns a safe help/fallback response; no arbitrary command is executed.
TC-12	Enable TLS and present a certificate not trusted by the server/client.	Handshake fails with a controlled error; connection is not accepted.
TC-13	PUT JSON/value to documented endpoint, then GET.	API returns documented success status and correct JSON/value response.
TC-14	Send multi-key PUT/DELETE operations across two shards to the dedicated `/api/v1/xtxn` endpoint.	Coordinator records and executes the cross-shard decision; all intended changes commit atomically, or all are aborted on conflict.
TC-15	Send a multi-key transaction spanning two shards to ordinary `/api/v1/txn`.	Gateway rejects the request before forwarding it to either shard; no key is partially changed.

Implementation review. The supplied `kvstore.zip` contains the HTTP API, TLS validation, operational AI query engine, metrics endpoints, shard gateway, and cross-shard two-phase-commit coordinator. These are implemented in the reviewed source. Matching automated tests include `TestHTTPKVRoundTrip`, `TestHTTPMetricsPromAndJSON`, `TestNLQueryEngineUnrecognizedFallsBackToHelp`, `TestClientRejectsUntrustedServerWithoutInsecureSkip`, `TestCrashAndRestartPreservesCommittedWrites`, `TestSnapshotIsolationWriteConflictFirstCommitterWins`, `TestCrossShardTxnCommitsAtomicallyOnAllShards`, and `TestGatewayRejectsCrossShardTxn`. The repository README reports the test suite passed previously; it could not be rerun in this environment because the required Go 1.25.0 toolchain download was blocked. "Implemented" therefore records code availability and test-suite coverage, not a newly generated pass result for every manual step.

5 Part B — Detailed Test Case Records

5.1 TC-01: Create a key and read it back (positive test)

Test case summary

Test ID	TC-01
Requirement ID	FR-05 (Put key), FR-04 (Get key)
Objective	Verify that a valid write creates or updates a key and that a subsequent read returns the exact stored value.
Test type	**Positive**
Priority	High
Status	**Implemented** (implementation and automated test coverage found in repository)

Preconditions

1. A test cluster is running and has an elected leader with a functioning quorum.
2. The client can reach at least one configured node.

3. The test key `lab9/course` does not contain a value from a previous test, or the test uses a unique key.

Input data

Field	Value
Key	`lab9/course`
Value	`MCA`
Operation sequence	PUT(key, value) followed by GET(key)

Steps to execute

1. Start the test cluster and confirm that a leader is elected.
2. Use the project's supported client interface to write key `lab9/course` with value `MCA`.
3. Record the write response and ensure that the operation is reported as committed/successful.
4. Issue a GET for the same key through the client.
5. Compare the returned value with the exact input value and save the terminal/API output as evidence.

Expected result (testable)

- The PUT operation returns a success/commit result.
- The GET operation returns the value `MCA` for key `lab9/course`.
- The returned value matches the input byte-for-byte; no unrelated key is modified.

Pass / fail rule Pass only if the write is committed and the subsequent GET returns exactly `MCA`. Mark Fail if the value differs, the operation reports success but the key is absent, or an unrelated key changes. Mark Blocked if the cluster cannot be started or the required client interface is unavailable.

Actual result / evidence:

Execution date / build:

5.2 TC-14: Commit a cross-shard transaction atomically (positive test)

Test case summary

Test ID	TC-14
Requirement ID	FR-73–FR-78 (Dedicated cross-shard transaction coordinator)
Objective	Verify that the dedicated `/api/v1/xtxn` endpoint atomically commits multi-key writes/deletes across independent shards using the two-phase-commit coordinator.
Test type	**Positive**
Priority	High
Status	**Implemented** (implementation and automated test coverage found in repository)

Preconditions

1. A build containing the shard gateway and at least two independent shard clusters is available.
2. The gateway's key-to-shard mapping can be queried or otherwise determined for test keys.
3. Keys `lab9/shardA` and `lab9/shardB` are confirmed to map to different shards.
4. Initial values (if any) are recorded so that unintended mutations can be detected.

Input data

```
Transaction operations:
  PUT lab9/shardA = "alpha"
  PUT lab9/shardB = "beta"

Precondition: shard(lab9/shardA) != shard(lab9/shardB)
Endpoint: POST /api/v1/xtxn
Expected: atomic commit on every participating shard
```

Steps to execute

1. Start the gateway and the shard clusters using the test configuration.
2. Confirm that the two test keys are assigned to different shard IDs.
3. Record the current values or confirm that both keys are absent.
4. Submit one transaction containing both PUT operations.
5. Capture the HTTP/API or gateway response, including status code and error message.
6. Read both keys from their respective shards and compare their state with the baseline.

Expected result (testable)

- The dedicated cross-shard endpoint accepts the transaction and coordinates the participating shards.
- The coordinator durably records its decision and resolves all participating shards consistently.
- Both writes become visible after commit; if a prepare/commit conflict is injected, neither shard is left with a partial transaction.
- The gateway and shard clusters remain healthy after the transaction.

Pass / fail rule Pass if the transaction outcome is consistent across all involved shards: all intended writes/deletes commit, or a conflict causes the transaction to abort without partial application. Mark Fail if the gateway reports success while only a subset of shards is updated, or if recovery leaves an unresolved partial outcome.

Actual result / evidence:

Execution date / build:

---

6 Requirement Traceability Matrix

Requirement traceability ensures that every selected requirement is linked to one or more tests. It also helps identify requirements that still need tests.

Req. ID	Requirement summary	Test case IDs	Coverage
FR-02	Leader election and single valid leader per term	TC-08	Positive
FR-03	Replication of committed operations	TC-09	Positive
FR-04	Read existing/missing keys	TC-01, TC-02, TC-03	Positive + negative outcome
FR-05	Create/update a key	TC-01, TC-13	Positive
FR-06	Delete a key	TC-03, TC-04	Positive + negative
FR-07	Single-shard multi-operation transaction	TC-05	Positive
FR-08	MVCC conflict detection	TC-06	Negative/conflict
FR-10	WAL durability and recovery	TC-07	Positive
FR-12	Leader discovery and retry	TC-08	Positive
FR-16	Prometheus metrics export	TC-10	Positive
FR-21	Safe natural-language query handling and restricted actions	TC-11	Negative
FR-22	Optional TLS/mTLS certificate validation	TC-12	Negative
FR-23	REST/JSON endpoints	TC-13	Positive
FR-72	Reject cross-shard use of ordinary transaction endpoint	TC-15	Negative
FR-73–FR-78	Dedicated atomic cross-shard transaction and recovery	TC-14	Positive + recovery/conflict coverage

6.1 Traceability checklist

☐ Each test case has a unique test ID.

☐ Each test is linked to at least one SRS requirement ID.

☐ Both positive and negative cases are included.

☐ Preconditions and input values are explicit.

☐ Expected results are observable and measurable rather than vague.

☐ Each test can be repeated by another tester using the same configuration.

☐ Implementation status is checked against the supplied repository; tests are only marked passed when a test execution result is available.

☐ No test is marked Pass unless it has actually been executed and evidence recorded.

7 Test Execution Record and Result

7.1 Execution summary template

The table below distinguishes source/test-suite evidence from a fresh execution in this report-preparation environment. The supplied repository README records a previously passing full test suite; a fresh rerun was blocked because the Go 1.25.0 toolchain could not be downloaded here.

Repository evidence reviewed

The source archive contains automated tests covering the relevant implementation areas:

- REST key operations and metrics export: `TestHTTPKVRoundTrip`, `TestHTTPMetricsPromAndJSON`.
- Safe natural-language query handling: `TestNLQueryEngineUnrecognizedFallsBackToHelp`.
- TLS trust validation: `TestClientRejectsUntrustedServerWithoutInsecureSkip`.
- MVCC conflict and durable restart recovery: `TestSnapshotIsolationWriteConflictFirstCommitterWins`, `TestCrashAndRestartPreservesCommittedWrites`.
- Cross-shard atomic commit and ordinary-endpoint rejection: `TestCrossShardTxnCommitsAtomicallyOnAllShards`, `TestGatewayRejectsCrossShardTxn`.

The project README reports a previously passing full test suite. The suite could not be rerun during this update because the Go 1.25.0 toolchain download was blocked in the current environment.

Total designed test cases	15
Positive test cases	9
Negative / conflict test cases	6
Freshly rerun in this environment	No (Go 1.25.0 download unavailable)
Repository evidence	README reports `go test ./... -race` passed previously
Build / commit tested	To be recorded
Test environment	Node count, OS, Go version, configuration

Result statement for submission

A functional test register was prepared for the `kvstore` project using the supplied SRS and a review of the uploaded implementation. It covers key-value operations, transactions, MVCC conflict handling, WAL recovery, leader discovery/retry, replication, metrics, safe natural-language queries, TLS validation, REST behavior, and both supported atomic cross-shard transactions and rejection of cross-shard operations sent to the ordinary transaction endpoint. The repository contains corresponding automated tests, and its README reports a previously passing full test suite. The current preparation environment could not independently rerun the suite because Go 1.25.0 was not installed and network access prevented toolchain download.

8 Conclusion

This lab translates KV Store requirements into repeatable functional checks and maps them to implementation evidence. The report records available test coverage without claiming a fresh test pass.